INFOSYS SPRINGBOARD 7.0 INTERNSHIP CAPSTONE PROJECT

# PricePilot AI: Software Requirements Specification

IEEE Std 830-1998 Compliant System Requirements Specification Document

| | |
|---|---|
| **Document Title:** | Software Requirements Specification (SRS) |
| **Standard Compliance:** | IEEE Std 830-1998 Specification Standard |
| **Document ID:** | DOC-SRS-PRICEPILOT-2026-V2 |
| **Project Release:** | Version 2.0.0 Enterprise Production |
| **Organization:** | Infosys Springboard 7.0 Internship Program |
| **Completion Date:** | August 2026 |
| **Authoring Team:** | Narendar Reddy, Manvitha, Pravallika, Ashwindh |
| **Technical Stack:** | FastAPI • React 19 • PostgreSQL • Extra Trees Regressor |

## 1.0 CERTIFICATE OF ACADEMIC COMPLETION

This is to certify that the project entitled **'PricePilot AI: Machine Learning-Based Dynamic Pricing & Demand Forecasting System'** is a bona fide record of capstone engineering work carried out by Narendar Reddy, Manvitha, Pravallika, and Ashwindh under the Infosys Springboard 7.0 Internship Program (August 2026).

| Role / Title | Name of Evaluator / Mentor | Signature & Date |
| --- | --- | --- |
| Infosys Project Mentor | Infosys Springboard Review Board | Approved (Digital Seal) |
| Lead Software Architect | Narendar Reddy | Verified (August 2026) |
| AI/ML Research Lead | Manvitha | Verified (August 2026) |
| Frontend UI/UX Specialist | Pravallika | Verified (August 2026) |
| Backend & DevOps Lead | Ashwindh | Verified (August 2026) |

## 2.0 ACKNOWLEDGEMENT

We express our profound gratitude to the **Infosys Springboard 7.0 Team** for providing the technical platform, mentoring resources, and guidance required to execute this project. Special appreciation goes to our academic advisors and industry reviewers whose rigorous feedback shaped the production architecture of PricePilot AI.

## 3.0 EXECUTIVE ABSTRACT

PricePilot AI is an enterprise-grade artificial intelligence dynamic pricing platform engineered to solve e-commerce price stagnation, gross margin erosion, and static list pricing. By integrating machine learning regression models, real-time demand forecasting, and structured analytical reporting, PricePilot AI empowers online retailers to optimize pricing dynamically based on product dimensions, shipping freight value, temporal demand patterns, and cost structures.

The platform features an **Extra Trees Regressor model** achieving a benchmarked **96.50% $R^2$ Score** with sub-50ms inference latency, a decoupled FastAPI REST backend, a Neon Cloud PostgreSQL database, an openpyxl Excel exporter generating styled workbooks, and a responsive glassmorphic React 19 single-page application.

# TABLE OF CONTENTS

## 4.0 DOCUMENT REVISION HISTORY

| Rev # | Date | Author | Reviewer | Approver | Description of Changes |
|---|---|---|---|---|---|
| 1.0 | 2026-08-01 | Narendar R. | Manvitha | Infosys Mentor | Initial Baseline Requirements Specification |
| 1.5 | 2026-08-04 | Pravallika | Ashwindh | Technical Lead | Added ML Benchmarking & Security Constraints |
| 2.0 | 2026-08-07 | Team PricePilot | QA Committee | Academic Board | Final IEEE 830 Production Release Validation |

## 5.0 INTRODUCTION & PROBLEM RATIONALE

E-commerce enterprises operate in highly dynamic markets characterized by fluctuating demand, competitive price updates, inflation spikes, and complex shipping freight cost calculations. Traditional pricing strategies reliance on static cost-plus markups or manual competitor monitoring leads to significant gross margin erosion and inventory stagnation.

PricePilot AI resolves these challenges through an artificial intelligence-driven dynamic pricing platform. By combining multi-variable regression models with high-throughput REST APIs and interactive dashboards, PricePilot AI automates pricing decisions with 96.50% $R^2$ accuracy.

## 6.0 DOCUMENT PURPOSE & AUDIENCE

This document specifies the complete software requirements for PricePilot AI Enterprise Platform version 2.0.0 adhering strictly to the IEEE Std 830-1998 standard. It provides a formal reference contract for software developers, AI engineers, system testers, database architects, and academic evaluators.

## 7.0 SYSTEM SCOPE & DELIVERABLES

The scope of PricePilot AI encompasses:

1. **AI Price Prediction Engine:** Scikit-Learn Extra Trees model executing real-time pricing regression in under 50ms.
2. **OAuth2 & Security Subsystem:** JWT authentication, Bcrypt password hashing (12 rounds), and 6-digit OTP verification.
3. **Admin User Registry:** Status approval workflows, role editing, and native `.xlsx` openpyxl Excel exports.
4. **Documentation Portal:** Dynamic serving of 24 publication-grade PDF documents directly through REST endpoints.

## 8.0 DEFINITIONS, ACRONYMS & ABBREVIATIONS

| Term / Acronym | Full Expansion | Definition & Context in Project |
|---|---|---|
| SRS | Software Requirements Specification | Formal IEEE 830 requirements specification document. |

| | | |
|---|---|---|
| JWT | JSON Web Token | Cryptographically signed bearer token for user authentication (HS256). |
| OTP | One-Time Password | 6-digit numerical code used for secure account password recovery. |
| RBAC | Role-Based Access Control | Authorization system enforcing Admin vs User access privileges. |
| MAE | Mean Absolute Error | Evaluation metric measuring average prediction discrepancy in Rupees. |
| RMSE | Root Mean Squared Error | Standard deviation of prediction residual errors. |
| $R^2$ Score | Coefficient of Determination | Statistical measure of how well regression predictions approximate real data. |

## 9.0 OVERALL SYSTEM DESCRIPTION

PricePilot AI operates as a self-contained, multi-tenant SaaS application. Users access the application via a web browser to perform real-time price predictions, view historical analytics, and manage accounts. The system interacts with a PostgreSQL database and serialized machine learning binary models.

## 10.0 PRODUCT PERSPECTIVE & 4-TIER ARCHITECTURE

The platform adopts a decoupled 4-tier SaaS architecture designed for cloud scalability:

- **Tier 1 (Presentation):** React 19 SPA built with Vite, Tailwind CSS, and Framer Motion.
- **Tier 2 (API Gateway):** FastAPI web server with Uvicorn ASGI runner and CORS middleware.
- **Tier 3 (Analytics Engine):** Scikit-Learn Extra Trees Regressor executing predictions.
- **Tier 4 (Persistence):** Neon Serverless PostgreSQL storing users, predictions, and logs.

### Architecture Topology Blueprint:

```
React 19 SPA (Vite) ■■[HTTPS JSON]■■> FastAPI Gateway ■■> Extra Trees Engine & Neon PostgreSQL DB
```

## 11.0 HIGH-LEVEL PRODUCT FUNCTIONS

| Function Module | Primary Component | Functional Summary |
|---|---|---|
| F-01: Authentication | routers/auth.py | JWT registration, Bcrypt hashing, login token generation, 6-digit OTP reset. |
| F-02: AI Pricing | routers/predict.py | Multi-variable Extra Trees ML price regression, margin calculation, confidence score. |
| F-03: User Admin | routers/users.py | Admin CRUD table, status toggle (pending/approved/blocked), openpyxl Excel export. |
| F-04: System Docs | routers/docs.py | Dynamic REST endpoints serving 24 publication-grade PDF and PPTX files. |

## 12.0 USER CLASSES & CHARACTERISTICS

| User Class | Privilege Level | Technical Expertise | System Access Rights |
|---|---|---|---|
| System Administrator | Elevated (Admin) | High | Full CRUD on users, Excel export, system stats, document management. |
| Retail Pricing Analyst | Standard User | Moderate | Execute predictions, view analytics dashboard, access prediction history. |
| General E-commerce User | Standard User | Basic | Execute price predictions, manage personal profile and settings. |
| Unauthenticated Guest | Public Access | Basic | Register account, execute login, request password reset OTP. |

## 13.0 OPERATING ENVIRONMENT & DEPENDENCIES

The system operates in the following hardware and software environment:

- **Server OS:** Ubuntu 22.04 LTS / Windows Server 2022 (x86_64 / ARM64).
- **Python Runtime:** Python 3.13.1 with Virtualenv (`venv`).
- **Node.js Runtime:** Node.js v20.11.0 LTS with `npm` 10+.
- **Database Server:** PostgreSQL 16 serverless cloud database (Neon Cloud) / SQLite local fallback.
- **Containerization:** Docker 24.0+ with Docker Compose v2.

## 14.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

| Req ID | Module | Functional Requirement Specification Statement | Priority |
|---|---|---|---|
| FR-1.1 | Auth | System shall authenticate users using OAuth2 Bearer JWT tokens with 12-hour expiration. | High |
| FR-1.2 | Auth | System shall hash all user passwords using Bcrypt algorithm with 12 work factor rounds. | Critical |
| FR-1.3 | Auth | System shall issue 6-digit OTP codes valid for 15 minutes upon password reset request. | High |
| FR-2.1 | ML Engine | System shall compute price predictions using pre-trained Extra Trees Regressor model. | Critical |
| FR-2.2 | ML Engine | System shall return predicted price, profit margin %, estimated cost, and confidence score. | High |
| FR-3.1 | Admin CRUD | System shall allow admins to approve, block, update roles, and delete user accounts. | High |
| FR-3.2 | Excel Export | System shall compile native openpyxl .xlsx reports featuring custom styling and freeze panes. | Critical |
| FR-4.1 | Docs Hub | System shall serve binary PDF and PPTX documentation files via REST API endpoints. | Medium |

## 15.0 NON-FUNCTIONAL REQUIREMENTS (NFR METRICS)

| NFR Category | Target Quality Metric | Measurement & Verification Method |
|---|---|---|
| Performance | Sub-50ms ML prediction latency per request | Benchmark via Locust load testing & Uvicorn logs. |
| Throughput | Support minimum 500 concurrent REST requests/sec | Stress testing with 500 virtual worker threads. |
| Availability | 99.95% operational uptime SLA | Automated health checks via `/api/health` endpoint. |
| Security | Zero plaintext passwords in DB; OWASP Top 10 compliance | Bcrypt hashing verification & automated security scan. |
| Usability | Responsive SPA layout rendering within 1.0 second | Google Lighthouse audit score >= 92. |

## 16.0 USE CASE SPECIFICATIONS (UC-01 TO UC-10)

| Use Case ID | Use Case Title | Primary Actor | Main Flow Description |
|---|---|---|---|
| UC-01 | User Registration | Guest User | Input credentials -> Bcrypt hash password -> Insert to DB as pending. |
| UC-02 | User Login | Registered User | Validate email/password -> Generate JWT access token -> Redirect to Dashboard. |
| UC-03 | Request Password Reset | User | Submit email -> Generate 6-digit OTP -> Save to DB -> Return success. |
| UC-04 | Verify Reset OTP | User | Input 6-digit OTP -> Verify expiry -> Update user password hash -> Flag OTP used. |
| UC-05 | Execute Price Prediction | User / Admin | Input 16 product features -> Run Extra Trees model -> Display price & margin. |
| UC-06 | View Analytics Dashboard | User / Admin | Fetch system stats -> Render volume charts and historical prediction logs. |
| UC-07 | Manage Users Table | Admin | Fetch all users -> Toggle status (approve/block) -> Save DB commit. |
| UC-08 | Export Users to Excel | Admin | Click Export -> Query DB -> Compile openpyxl workbook -> Stream .xlsx binary. |
| UC-09 | Explore Project Docs | User / Admin | Fetch document list -> Preview metadata -> Download PDF/PPTX binary. |
| UC-10 | Update Profile Settings | User | Edit name/phone/avatar -> Validate inputs -> Commit update to PostgreSQL DB. |

# 17.0 SYSTEM ACTIVITY DIAGRAMS & WORKFLOWS

The Activity Diagram below illustrates the flow of execution during an AI Price Prediction request:

```
User Submits Form Data ■■> FastAPI Validates Pydantic Schema ■■> Verify JWT Token ■■> Extra Trees Inference
■■> Save DB Log ■■> Return Response JSON
```

# 18.0 DATA FLOW DIAGRAMS (DFD LEVEL 0, 1, 2)

## DFD Level 0 (Context Diagram):

```
[User / Admin] ■■< HTTP Requests >■■> (0.0 PricePilot AI System) ■■< JSON / XLSX / PDF >■■> [User / Admin]
```

## DFD Level 1 (Subsystem Breakdown):

```
(1.0 Auth Subsystem) ■■> D1: Users DB Table (2.0 Prediction Subsystem) ■■> D2: Predictions DB Table & ML
Engine (.pkl) (3.0 Admin Subsystem) ■■> D1: Users DB Table & openpyxl Exporter
```

# 19.0 ENTITY RELATIONSHIP (ER) DIAGRAM & DATA DICTIONARY

| Table Name | Primary Key | Foreign Keys | Index Columns | Functional Purpose |
|---|---|---|---|---|
| users | id (INT) | None | email, username | Stores user account credentials, status, and roles. |
| predictions | id (INT) | product_id, user_id | created_at | Logs ML price prediction results. |
| products | id (INT) | None | category, name | Catalog items and cost structures. |
| password_reset_otps | id (INT) | user_id | email_or_phone, otp_code | Stores 6-digit expiring password reset codes. |
| activity_logs | id (INT) | user_id | timestamp | Audit log tracking system operations. |

## 20.0 SYSTEM CONSTRAINTS & ARCHITECTURAL BOUNDARIES

1. **Model Serialization Constraint:** The Extra Trees model file size (~815MB) requires sufficient server RAM (minimum 2GB) during startup.
2. **Database Lock Constraints:** SQLite local fallback locks database during concurrent writes; production requires Neon PostgreSQL.
3. **Token Expiry Constraint:** JWT access tokens expire after 12 hours requiring client re-authentication.

## 21.0 ASSUMPTIONS & SYSTEM DEPENDENCIES

• System assumes active internet connectivity for Neon Cloud PostgreSQL database connections.
• Client devices are assumed to use modern HTML5/ES6 compliant web browsers (Chrome, Edge, Firefox, Safari).

## 22.0 SECURITY REQUIREMENTS & OWASP MITIGATION

| Threat Classification | OWASP ASVS Standard | Security Control Implemented |
|---|---|---|
| SQL Injection | V5: Input Validation | SQLAlchemy Parameterized Query Execution. |
| Credential Hashing | V2: Authentication | Bcrypt Hashing with 12 Work Factor Rounds. |
| Session Hijacking | V3: Session Management | OAuth2 Bearer JWT Tokens (HS256 Encryption). |
| Cross-Origin Attacks | V14: Configuration | Security Headers Middleware (X-Frame-Options, CSP, CORS). |

## 23.0 PERFORMANCE REQUIREMENTS & LATENCY BENCHMARKS

• ML Prediction Latency: Average 45 ms per item prediction.
• Database Query Speed: Average 12 ms per SQL query.
• Frontend Vite Build Time: 908 ms complete build time.

## 24.0 DATABASE REQUIREMENTS & INDEXING STRATEGY

PostgreSQL database uses explicit indexes on `users.email`, `users.username`, `predictions.created_at`, and `password_reset_otps.email_or_phone` to guarantee sub-15ms query lookups.

## 25.0 EXTERNAL INTERFACE REQUIREMENTS

1. **User Interfaces:** React 19 glassmorphic interface with dark mode theme.
2. **API Interfaces:** REST JSON endpoints with HTTP status codes 200, 201, 400, 401, 403, 404, 500.
3. **Software Interfaces:** Openpyxl Excel engine, ReportLab PDF generator, Python 3.13.

## 26.0 MACHINE LEARNING BENCHMARK METRICS

| Model Name | $R^2$ Score | MAE (■) | RMSE (■) | Inference Speed | Selection Rationale |
|---|---|---|---|---|---|
| Extra Trees Regressor | 0.9650 | 12.40 | 18.60 | 0.045 s | Selected (Best $R^2$ & lowest MAE) |
| Random Forest Regressor | 0.9420 | 15.80 | 22.10 | 0.082 s | Evaluated Baseline |
| XGBoost Regressor | 0.9380 | 16.20 | 23.40 | 0.038 s | Evaluated Baseline |
| Gradient Boosting | 0.9150 | 19.50 | 27.80 | 0.055 s | Evaluated Baseline |
| Decision Tree | 0.8840 | 24.10 | 34.20 | 0.012 s | Evaluated Baseline |
| Linear Regression | 0.7410 | 42.50 | 58.90 | 0.005 s | Evaluated Baseline |

## 27.0 SOFTWARE TESTING & QA REPORT

All API endpoints and authentication workflows were tested using automated Pytest scripts achieving 100% test pass rate across 54 unit test cases.

## 28.0 FUTURE ENHANCEMENTS & ROADMAP

Planned post-release capabilities include real-time competitor web scraping, multi-currency conversion engines, automated monthly model re-training pipelines, and native iOS/Android mobile applications.

## 29.0 REFERENCES & STANDARDS

1. IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
2. FastAPI Documentation: https://fastapi.tiangolo.com/
3. Scikit-Learn Documentation: https://scikit-learn.org/
4. OWASP Application Security Verification Standard (ASVS v4.0).

## 30.0 APPENDIX & CORE SOURCE CODE LISTINGS

### Core Backend Application Entry Point (`backend/main.py`):

```
from fastapi import FastAPI from routers import auth, predict, users, dashboard, docs app =
FastAPI(title='PricePilot AI Enterprise Platform', version='2.0.0')
```